



EGE UNIVERSITY

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

Artificial Intelligence Methods (3+0)

2025–2026 SPRING SEMESTER

PROJECT-1 REPORT

DELIVERY DATE

18/04/2026

PREPARED BY

Lidya Deliağa , 05220000366

Ceylin Tor , 05230000350

Aykut Hacıoğlu , 05220001067

Ali Batıkan Öcal , 05230000983

Ömer Faruk Çakmakçı , 05220000991

Index

1) Algorithms, Definitions, Comparison, Research and Comment	4
1.a A* and DE Algorithm	4
1.a.i Image of A* Algorithm Written on a Paper (Scan/Photograph)	4
1.a.ii Comparison of A* and DE Algorithms	5
1.b 2D or 3D Computer Game and GenAI	5
1.b.i Making Two or Three Dimensional Computer Game Code to GenAI tool	6
1.b.ii Execution of The Code and Test Screenshots	12
1.b.iii Commands You Write to Add New Features and Added Features	12
1.b.iv Explaining the Advantages & Disadvantages of Using Gen AI in Software Development...	17
1.c Definitions and Comparisons	17
1.c.i GPT (Generative Pre-trained Transformer)	18
1.c.ii Gradient Descent	18
1.c.iii Nonlinear Problem	18
1.c.iv GAN (Generative Adversarial Network) vs Diffusion Models	19
1.c.v Decision Tree vs Neural Network models	19
1.d GenAI and Job Applications	19
1.d.i Answer the question in your own words on a paper (Scan/Photograph)	21
1.d.ii Ask the generative AI the question in reasoning mode and write down what you understand from its answer	22
// Explanation	23
2) Problem Solving and Coding	23
2.a Solving the Eight Queen Problem with First-Choice Hill Climbing / Solving the Bridge and Torch Problem with A*	23
2.a.i Codes and Outputs	23
2.a.ii What is Stochastic Hill Climbing? Explain which part of the Hill Climbing source code needs to be changed.	29
3) Genetic Algorithms	30
3.a Generated Questions and Your Answers	30
4) Machine Learning with Python [Classifier]	34
4.a Python Standardization Example	34

4.a.i Codes.....	34
4.a.ii Difference from Normalization	34
4.b Finding or Creating Dataset	34
4.c Review the Dataset	34
4.d Two different classifier and results.....	35
4.d.i Source Code	35
4.d.ii Confusion Matrices	36
4.e Predicting the class of an instance that its class is unknown.....	36
4.e.i Source Code	36
4.e.ii Output	37
4.f 3-Fold Cross Validation	37
4.f.i Source Code	37
4.f.ii Classification Performance with 3-Fold Cross Validation	38
5) Time Spent and Division of labor Details	39
6) Self-assessment Table	39

1) Algorithms, Definitions, Comparison, Research and Comment

1.a A* and DE Algorithm

1.a.i Image of A* Algorithm Written on a Paper (Scan/Photograph)

A* Algorithm

Input :
start node
goal node

open-list $\leftarrow \{ \text{start} \}$
closed-list $\leftarrow \emptyset$

$g(\text{start}) \leftarrow 0$
 $h(\text{start}) \leftarrow \text{heuristic}(\text{start}, \text{goal})$
 $f(\text{start}) \leftarrow g(\text{start}) + h(\text{start})$

while open-list is not empty do
 current $\leftarrow$ node in open-list with smallest f value
 if current = goal then
 return path from start to goal
 remove current from open-list
 add current to closed-list
 for each neighbor of current do
 if neighbor is in closed-list then
 continue
 tentative-g $\leftarrow g(\text{current}) + \text{cost}(\text{current}, \text{neighbor})$
 if is not in open-list then
 add neighbor to open-list
 else if tentative-g $\geq g(\text{neighbor})$ then
 continue
 parent(neighbor) $\leftarrow$ current
 $g(\text{neighbor}) \leftarrow$ tentative-g
 $h(\text{neighbor}) \leftarrow \text{heuristic}(\text{neighbor}, \text{goal})$
 $f(\text{neighbor}) \leftarrow g(\text{neighbor}) + h(\text{neighbor})$
return failure

$$f(n) = g(n) + h(n)$$

$g(n)$: the actual cost from the start node to node n

$h(n)$: the estimated cost from node n to the goal

$f(n)$: the total estimated cost of the solution path through node n

1.a.ii Comparison of A* and DE Algorithms

A* (A-star) and Differential Evolution (DE) are fundamentally different algorithms, both in terms of how they work and the types of problems they are designed to solve. A* is a search algorithm mainly used to find the shortest or least-cost path between a start and a goal state. It is widely applied in areas such as pathfinding, robotics, and game development. The algorithm evaluates each node using the function $f(n)=g(n)+h(n)$, where $g(n)$ represents the actual cost from the start node to the current node, and $h(n)$ is an estimate of the remaining cost to reach the goal. This combination allows A* to efficiently guide the search toward the target, especially when a good heuristic is used.

On the other hand, Differential Evolution (DE) is an optimization algorithm rather than a search algorithm. It is designed to solve continuous optimization problems by iteratively improving a population of candidate solutions. Instead of working with nodes and paths, DE operates on vectors and applies operations such as mutation, crossover, and selection to generate better solutions over time. It is particularly useful for problems where the solution space is large, nonlinear, or difficult to model analytically.

The key difference between the two lies in their purpose. A* focuses on finding an optimal path within a defined state space, while DE aims to find the best possible solution to an optimization problem. Their approaches also differ significantly: A* follows a deterministic and goal-directed strategy, expanding one node at a time based on cost estimates, whereas DE is stochastic and works with multiple candidate solutions simultaneously.

In terms of output, A* produces a sequence of steps (a path) from the start state to the goal, while DE returns the best solution vector it has found after several iterations. A* performs best when the problem can be modeled as a graph with a clearly defined goal, whereas DE is more suitable for tuning parameters or solving complex mathematical optimization problems.

In summary, A* is a structured search algorithm designed for pathfinding, while DE is a population-based optimization method aimed at exploring large solution spaces and finding near-optimal solutions. Both are effective in their own domains, but they are not interchangeable due to their fundamentally different objectives and working principles.

1.b 2D or 3D Computer Game and GenAI

1.b.i Making Two or Three Dimensional Computer Game Code to GenAI tool

PROMPT: Create a simple 2D endless runner game using Python and Pygame.

Game requirements:

- Use pygame library
- The player is a square that automatically moves forward.
- The player can press the SPACE key to invert gravity (switch between floor and ceiling).
- There should be obstacles coming from the right side of the screen.
- If the player collides with an obstacle, the game ends.
- The game should include a score system based on survival time.
- Display the score on the screen.
- Use simple shapes (rectangles) for player and obstacles (no images).
- Keep the code clean, readable, and in a single Python file.
- Include comments explaining the important parts of the code.
- Window size: around 800x600

Just do what I tell you. If you need to do anything outside of what I've said, ask for my approval, and when you're finished, briefly describe the project to me. If you have any suggestions before we begin, let's discuss them first, then we can start developing it.

GEN AI: Suggestions & Considerations

Before I write the code, I'd like your approval or thoughts on these three small "quality of life" details:

Obstacle Variety: Should the obstacles be a fixed height, or would you like them to vary randomly (e.g., some tall, some short) to make the gravity switching more challenging?

Difficulty Scaling: Do you want the obstacles to speed up as the score increases, or should the speed remain constant?

Restart Mechanic: Usually, in these games, pressing 'R' or 'Space' after a game-over restarts the loop. Would you like me to include a simple "Game Over" screen with a restart option, or should the program simply close?

Would you like me to proceed with the code now, or would you like to adjust any of these points first?

PROMPT: Proceed with the code now.

Use fixed obstacle sizes, constant speed, and do not add any restart mechanic or extra features.

Follow only the original requirements and keep the game as simple as possible.

GEN AI: GENERATED CODE

```
import pygame

import random

import sys

# --- Configuration & Constants ---

SCREEN_WIDTH = 800

SCREEN_HEIGHT = 600

FPS = 60

# Colors

WHITE = (255, 255, 255)

BLACK = (0, 0, 0)

RED = (255, 0, 0)

BLUE = (0, 120, 255)

# Game Settings

PLAYER_SIZE = 40

OBSTACLE_WIDTH = 30

OBSTACLE_HEIGHT = 60

GAP_HEIGHT = 150 # Distance between floor and ceiling
```

```

SPEED = 7

SPAWN_RATE = 60 # Frames between obstacle spawns

class GravityRunner:

    def __init__(self):

        pygame.init()

        self.screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))

        pygame.display.set_caption("Gravity Runner")

        self.clock = pygame.time.Clock()

        self.font = pygame.font.SysFont("Arial", 32)

        # Player state

        self.player_rect = pygame.Rect(100, SCREEN_HEIGHT - 100, PLAYER_SIZE, PLAYER_SIZE)

        self.gravity_up = False # False = Floor, True = Ceiling

        # Environment

        self.floor_y = SCREEN_HEIGHT - 50

        self.ceiling_y = 50

        self.obstacles = []

        self.spawn_timer = 0

        # Score

        self.score = 0

        self.start_time = pygame.time.get_ticks()

    def spawn_obstacle(self):

        """Creates an obstacle on either the floor or the ceiling."""

```

```
on_top = random.choice([True, False])

if on_top:
    y_pos = self.ceiling_y
else:
    y_pos = self.floor_y - OBSTACLE_HEIGHT

new_obstacle = pygame.Rect(SCREEN_WIDTH, y_pos, OBSTACLE_WIDTH, OBSTACLE_HEIGHT)
self.obstacles.append(new_obstacle)
```

```
def run(self):
    running = True
    while running:
        # 1. Event Handling
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE:
                    self.gravity_up = not self.gravity_up

        # 2. Movement & Gravity Logic
        if self.gravity_up:
            self.player_rect.top = self.ceiling_y
        else:
            self.player_rect.bottom = self.floor_y

        # 3. Obstacle Logic
```

```

self.spawn_timer += 1

if self.spawn_timer >= SPAWN_RATE:

    self.spawn_obstacle()

    self.spawn_timer = 0


for obs in self.obstacles[:]:

    obs.x -= SPEED

    # Remove off-screen obstacles

    if obs.right < 0:

        self.obstacles.remove(obs)


    # Collision Detection

    if self.player_rect.colliderect(obs):

        running = False # Game Ends


# 4. Score Calculation (Survival Time)

self.score = (pygame.time.get_ticks() - self.start_time) // 100


# 5. Drawing

self.screen.fill(WHITE)


# Draw Floor and Ceiling boundaries

pygame.draw.line(self.screen, BLACK, (0, self.floor_y), (SCREEN_WIDTH, self.floor_y), 2)

pygame.draw.line(self.screen, BLACK, (0, self.ceiling_y), (SCREEN_WIDTH, self.ceiling_y), 2)


# Draw Player

pygame.draw.rect(self.screen, BLUE, self.player_rect)

```

```

    # Draw Obstacles

    for obs in self.obstacles:

        pygame.draw.rect(self.screen, RED, obs)

    # Draw Score

    score_text = self.font.render(f"Score: {self.score}", True, BLACK)

    self.screen.blit(score_text, (20, 10))

    pygame.display.flip()

    self.clock.tick(FPS)

pygame.quit()

sys.exit()

if __name__ == "__main__":

    game = GravityRunner()

    game.run()

```

Project Description

This project is a minimalist 2D Endless Runner built using the Pygame framework.

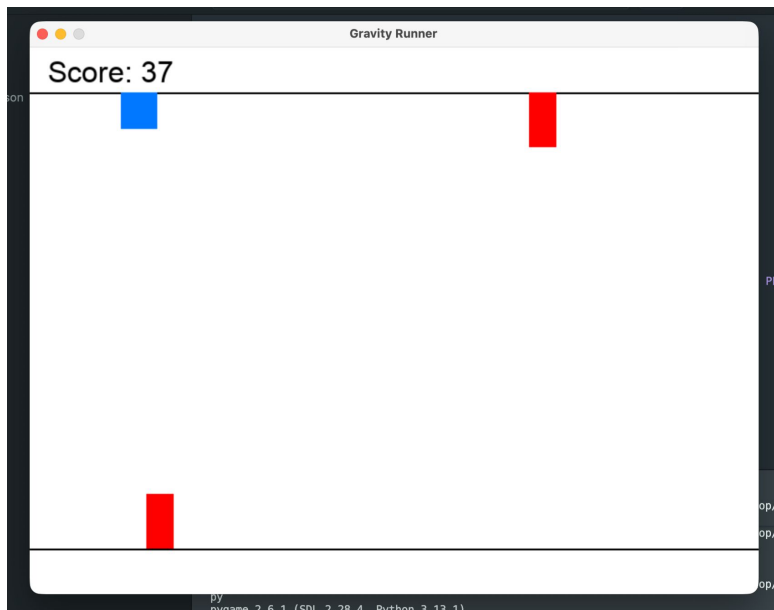
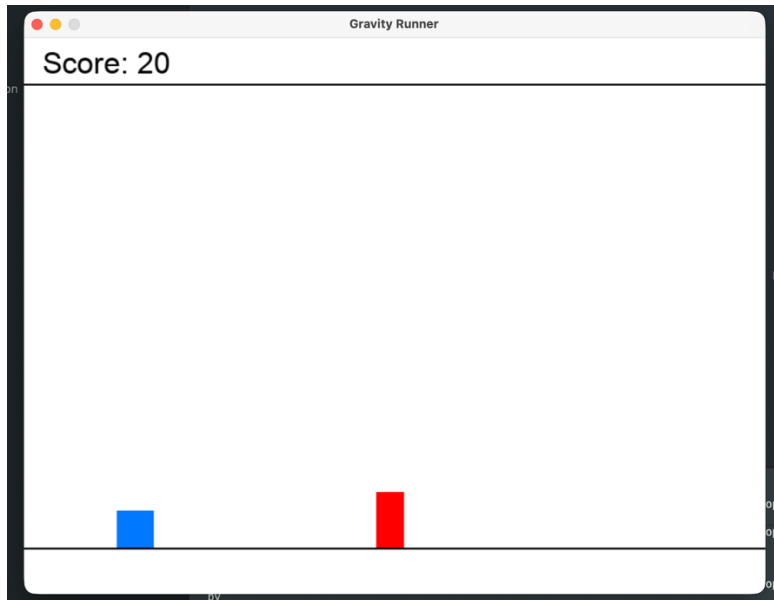
Mechanics: It features a "Gravity Inversion" system where the player (a blue square) toggles between a floor and ceiling boundary using the Spacebar.

Obstacles: Red rectangular obstacles spawn at a fixed rate and move at a constant speed from right to left.

Collision & Scoring: The game uses Pygame's built-in `colliderect` for hit detection. The score is calculated based on the elapsed time since the game started.

Architecture: The code is encapsulated in a single class to manage the game state, loop, and rendering efficiently within one file.

1.b.ii Execution of The Code and Test Screenshots



1.b.iii Commands You Write to Add New Features and Added Features

PROMPT:

Improve the existing Python Pygame by adding the following features. Do not change the base game.

New features to implement:

1. Color System:

- Add collectible star objects.
- When the player collects a star, the player color should change randomly.
- Use simple bright colors (e.g. orange, green, purple, blue).

2. Health System:

- Add a lives system.
- Player starts with 3 lives and maximum lives is 3.
- Add collectible heart objects.
- If the player collects a heart and already has 3 lives, do nothing.
- When the player collides with an obstacle:
 - If lives > 0, decrease lives by 1 and continue the game.
 - If lives == 0, end the game.

3. Restart Mechanic:

- When the game ends, display "Game Over" on the screen.
- Allow restarting the game by pressing the R key.
- Reset all variables properly (score, obstacles, lives, etc.).

4. Difficulty Increase:

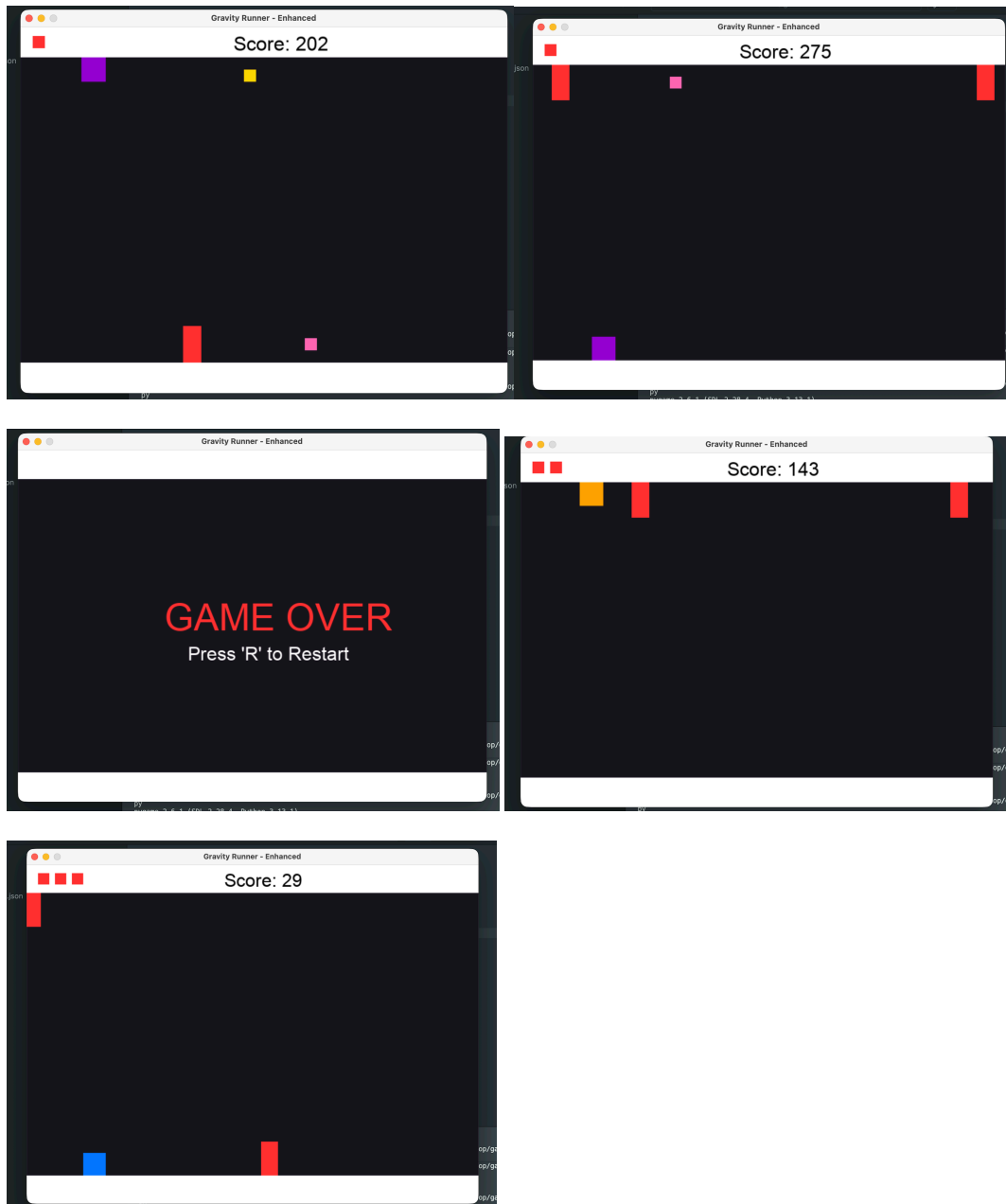
- Gradually increase obstacle speed over time based on score.
- Keep it simple (small incremental increase).

5. UI Improvements:

- Display lives at the top left using simple shapes (small red rectangles or hearts).
- Display score clearly on the screen.
- Use a slightly darker background color instead of pure white.
- Keep everything simple (no images, no complex animations).
- Keep everything in a single Python file.
- Use only pygame and basic shapes.
- Do not add advanced systems or unnecessary complexity.
- Keep the code clean and readable, with comments.

give phyton code only

SCREENSHOT:



EXPLANATION:

1. Dynamic Color System (Stars)

Mechanic: Yellow (GOLD) square objects have been added to the game world.

Logic: When a square from the `self.stars` list collides with the player (`colliderect`), a random color is selected from the `PLAYER_COLORS` list. The player's color updates instantly, providing visual feedback and breaking the monotony of the gameplay.

2. Advanced Health System (Hearts)

Mechanic: Pink (`HEART_CLR`) heart objects and a 3-life system.

Logic: Previously, the game ended on a single hit. Now, the `self.lives` variable manages survival:

Obstacle Collision: If the player hits a red rectangle, lives decreases by 1, and the obstacle is removed so it doesn't hit the player twice.

Heart Collection: If the player collects a heart and has fewer than 3 lives (if `self.lives < 3`), they gain 1 life. This introduces a risk-reward balance—deciding whether to risk a collision to grab a life-saving heart.

3. Difficulty Scaling

Mechanic: The game speed increases as the score goes up.

Logic: The line `current_speed = self.base_speed + (self.score // 50)` is the engine of the game's difficulty. For every 50 points scored, the movement speed increases by 1 unit. This ensures the game remains challenging and continues to test the player's reflexes over time.

4. Enhanced UI (User Interface)

Mechanic: On-screen Score and Life indicators.

Logic: * The `for i in range(self.lives)` loop draws small red rectangles in the top-left corner to visualize remaining health.

Since the background is dark, the score and lives are placed on the white "boundary" areas (ceiling and floor) to ensure maximum readability and a clean look.

5. Restart Logic

Mechanic: Resetting the game state using the 'R' key.

Logic: The `reset_game()` function clears all active lists (`self.obstacles`, `self.stars`, `self.hearts`), resets the speed to its base value, and sets the `self.game_active` flag back to `True`. This allows for seamless transitions between games without needing to restart the Python script.

GENERATED CODE:

```

main.py x
main.py % GravityRunner > update
1 import pygame
2 import random
3 import sys
4
5 # --- Configuration & Constants ---
6 SCREEN_WIDTH = 800
7 SCREEN_HEIGHT = 600
8 FPS = 60
9
10 # Colors
11 BACKGROUND = (25, 20, 30)
12 WHITE = (255, 255, 255)
13 BLACK = (0, 0, 0)
14 RED = (255, 50, 50)
15 GOLD = (255, 215, 0)
16 HEART_CLR = (255, 180, 180)
17 PLAYER_COLORS = ((0, 120, 255), (50, 200, 50), (140, 0, 211), (255, 165, 0))
18
19 # Game Settings
20 PLAYER_SIZE = 40
21 OBSTACLE_WIDTH = 30
22 OBSTACLE_HEIGHT = 60
23 COLLECTIBLE_SIZE = 20
24 FLOOR_Y = SCREEN_HEIGHT - 50
25 CEILING_Y = 50
26
27 class GravityRunner:
28     def __init__(self):
29         pygame.init()
30         self.screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
31         pygame.display.set_caption("Gravity Runner - Enhanced")
32         self.clock = pygame.time.Clock()
33         self.font = pygame.font.SysFont("Arial", 32)
34         self.large_font = pygame.font.SysFont("Arial", 64)
35         self.reset_game()
36
37     def reset_game(self):
38         """Initializes or resets all game variables."""
39         self.player_rect = pygame.Rect(100, FLOOR_Y - PLAYER_SIZE, PLAYER_SIZE, PLAYER_SIZE)
40         self.player_color = PLAYER_COLORS[0]
41         self.gravity_up = False
42
43         self.obstacles = []
44         self.stars = []
45         self.hearts = []
46
47         self.spawn_timer = 0
48         self.score = 0
49         self.lives = 3
50         self.base_speed = 7
51         self.game_active = True
52         self.start_ticks = pygame.time.get_ticks()
53
54     def spawn_entities(self):
55         """Handles spawning logic for obstacles and collectibles."""
56         self.spawn_timer += 1
57         if self.spawn_timer >= 60: # Spawn every 1 second approx
58             self.spawn_timer = 0
59
60         # 1. Spawn Obstacle
61         on_top = random.choice([True, False])
62         y_obs = CEILING_Y if on_top else FLOOR_Y - OBSTACLE_HEIGHT
63         self.obstacles.append(pygame.Rect(SCREEN_WIDTH, y_obs, OBSTACLE_WIDTH, OBSTACLE_HEIGHT))
64
65         # 2. Randomly spawn Star (30% chance)
66         if random.random() < 0.3:
67             y_star = CEILING_Y + 20 if not on_top else FLOOR_Y - 40
68             self.stars.append(pygame.Rect(SCREEN_WIDTH + 100, y_star, COLLECTIBLE_SIZE, COLLECTIBLE_SIZE))
69
70         # 3. Randomly spawn Heart (15% chance)
71         if random.random() < 0.15:
72             y_heart = CEILING_Y + 20 if on_top else FLOOR_Y - 40
73             self.hearts.append(pygame.Rect(SCREEN_WIDTH + 200, y_heart, COLLECTIBLE_SIZE, COLLECTIBLE_SIZE))
74
75     def update(self):
76         """Updates game logic, movement, and collisions."""
77         if not self.game_active:
78             return
79
80         # Difficulty: Increase speed based on score
81         current_speed = self.base_speed + (self.score // 50)
82
83         # Gravity Toggle
84         if self.gravity_up:
85             self.player_rect.top = CEILING_Y
86         else:
87             self.player_rect.bottom = FLOOR_Y
88
89         # Move and check Obstacles
90         for obs in self.obstacles[:]:
91             obs.x -= current_speed
92             if obs.colliderect(self.player_rect):
93                 self.lives -= 1
94                 self.obstacles.remove(obs)
95                 if self.lives <= 0:
96                     self.game_active = False
97             elif obs.right < 0:
98                 self.obstacles.remove(obs)
99
100         # Move and check Stars
101         for star in self.stars[:]:
102             star.x -= current_speed
103             if star.colliderect(self.player_rect):
104                 self.player_color = random.choice(PLAYER_COLORS)
105                 self.stars.remove(star)
106             elif star.right < 0:
107                 self.stars.remove(star)
108
109         # Move and check Hearts
110         for heart in self.hearts[:]:
111             heart.x -= current_speed
112             if heart.colliderect(self.player_rect):
113                 if self.lives < 3:
114                     self.lives += 1
115                 self.hearts.remove(heart)
116             elif heart.right < 0:
117                 self.hearts.remove(heart)
118
119         # Update Score
120         self.score = (pygame.time.get_ticks() - self.start_ticks) // 100

```

```

def draw(self):
    """Renders all elements to the screen."""
    self.screen.fill(BACKGROUND)

    # Draw Floor/Ceiling
    pygame.draw.rect(self.screen, WHITE, (0, 0, SCREEN_WIDTH, CEILING_Y))
    pygame.draw.rect(self.screen, WHITE, (0, FLOOR_Y, SCREEN_WIDTH, SCREEN_HEIGHT - FLOOR_Y))

    if self.game_active:
        # Draw Player
        pygame.draw.rect(self.screen, self.player_color, self.player_rect)

        # Draw Entities
        for obs in self.obstacles:
            pygame.draw.rect(self.screen, RED, obs)
        for star in self.stars:
            pygame.draw.rect(self.screen, GOLD, star)
        for heart in self.hearts:
            pygame.draw.rect(self.screen, HEART_CLR, heart)

        # UI: Score
        score_txt = self.font.render(f"Score: {self.score}", True, BLACK)
        self.screen.blit(score_txt, (SCREEN_WIDTH // 2 - 50, 10))

        # UI: Lives (Small red rectangles)
        for i in range(self.lives):
            pygame.draw.rect(self.screen, RED, (20 + (i * 30), 15, 20, 20))
    else:
        # Game Over Screen
        msg = self.large_font.render("GAME OVER", True, RED)
        retry = self.font.render("Press 'R' to Restart", True, WHITE)
        self.screen.blit(msg, (SCREEN_WIDTH // 2 - 150, SCREEN_HEIGHT // 2 - 50))
        self.screen.blit(retry, (SCREEN_WIDTH // 2 - 110, SCREEN_HEIGHT // 2 + 30))

    pygame.display.flip()

def run(self):
    while True:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_SPACE and self.game_active:
                    self.gravity_up = not self.gravity_up
                if event.key == pygame.K_r and not self.game_active:
                    self.reset_game()

        if self.game_active:
            self.spawn_entities()
            self.update()

        self.draw()
        self.clock.tick(FPS)

if __name__ == "__main__":
    game = GravityRunner()
    game.run()

```

1.b.iv Explaining the Advantages & Disadvantages of Using Gen AI in Software Development

Generative AI is a tool that can significantly speed up the software development process. One of its biggest advantages is that it can generate code quickly instead of writing everything from scratch. This is especially useful for simple or repetitive tasks. In this project, for example, we were able to create the basic structure of the game in a short time.

Another advantage is that it helps with learning. It can generate example code and explain how it works, which makes it easier to understand new topics. This is very helpful for students because it supports both coding and learning at the same time.

It also makes prototyping easier. You can quickly turn an idea into working code and test it. This helps developers experiment more and improve their projects faster.

However, there are also some disadvantages. One important problem is that you may not fully understand the generated code. Even if the code works, not knowing how it works can cause problems later, especially when debugging or making changes.

Another issue is code quality. Gen AI does not always produce clean or efficient code. Sometimes it can include unnecessary parts or even mistakes, so the code should always be reviewed carefully.

There is also a risk of becoming too dependent on it. If you rely too much on AI, your own problem-solving and coding skills may not improve enough.

Finally, there can be security risks. AI-generated code may not always follow best practices or up-to-date standards, which can be a problem in real-world applications.

In conclusion, Gen AI is a very useful tool that can save time and support learning, but it should be used carefully. It is better to use it as an assistant, not as a complete replacement for your own knowledge.

1.c Definitions and Comparisons

1.c.i GPT (Generative Pre-trained Transformer)

GPT (Generative Pre-trained Transformer) is an artificial intelligence model developed for natural language processing (NLP) tasks. It is based on the Transformer architecture and is trained on large datasets in a pre-training phase.

The main idea behind GPT is predicting the next word in a sequence by using probability distributions. It uses an attention mechanism to understand relationships between words in a sentence. After training, the model can be applied to different tasks such as text generation, summarization, and question answering.

Sources

Vaswani et al., 2017 – Attention Is All You Need

Goodfellow et al., Deep Learning

1.c.ii Gradient Descent

Gradient Descent is an optimization algorithm used in machine learning to minimize a loss (error) function.

It works by calculating the gradient (derivative) of the function and updating the parameters in the direction that reduces the error the fastest. A parameter called the learning rate determines how big each update step is.

Basic steps:

Initialize parameters randomly

Compute the loss

Calculate the gradient

Update parameters

Repeat until the minimum error is reached

Sources

Bishop, C. – Pattern Recognition and Machine Learning

Goodfellow et al., Deep Learning

1.c.iii Nonlinear Problem

A nonlinear problem is a problem where the relationship between variables is not linear. This means there is no constant rate of change between inputs and outputs.

Examples include functions like x^2 , $\sin(x)$, and $\log(x)$.

Most real-world problems are nonlinear, which makes them more complex and harder to solve. These problems can contain multiple local minima, making optimization more challenging. Therefore, advanced methods such as neural networks or evolutionary algorithms are often used.

Sources

Bishop, C. – Pattern Recognition and Machine Learning

Goodfellow et al., Deep Learning

1.c.iv GAN (Generative Adversarial Network) vs Diffusion Models

GAN and Diffusion Models are both used for generating data such as images or text, but they work in very different ways.

GAN consists of two models: a Generator and a Discriminator. The Generator tries to create realistic data, while the Discriminator tries to distinguish between real and fake data. These two models compete with each other, which helps improve the quality of generated outputs. However, GANs can be unstable during training and may suffer from problems like mode collapse.

Diffusion Models, on the other hand, gradually add noise to data and then learn how to reverse this process to generate new data. They are generally more stable and can produce higher-quality results, but they require more computational power and take longer to train and generate outputs.

Comparison

GAN feels like a faster but riskier approach. When it works, it can generate results quickly, but training can easily become unstable. Diffusion models are slower, but they are more reliable and usually produce better quality outputs. So in practice, it's more like choosing between speed and stability.

Sources

Goodfellow, I. – Generative Adversarial Networks (2014)

Ho et al., 2020 – Denoising Diffusion Probabilistic Models

Goodfellow et al., Deep Learning

1.c.v Decision Tree vs Neural Network models

Decision Trees and Neural Networks are both used for classification and prediction, but they differ in how they work and how easy they are to understand.

Decision Trees split data using simple if-else rules, which makes them easy to interpret. You can clearly see how a decision is made. They are also relatively fast to train.

Neural Networks consist of multiple layers of interconnected neurons and can learn complex patterns in data. However, they are often considered a “black box” because it is difficult to understand how they make decisions. They also require more data and longer training time.

Comparison

Decision Trees are easier to understand and quicker to train, so they are more practical when transparency matters. Neural Networks, on the other hand, are more powerful for complex problems, but they take longer to train and it’s harder to explain how they reach a decision.

Sources

Bishop, C. – Pattern Recognition and Machine Learning

Goodfellow et al., Deep Learning

1.d GenAI and Job Applications

What has changed in terms of what needs to be done during and before job applications in the field of artificial intelligence following developments in the field of generative artificial intelligence?

1.d.i Answer the question in your own words on a paper (Scan/Photograph).

//Image

After the development of generative artificial intelligence, job applications in the AI field have changed a lot. In the past, it was mostly enough to know programming, machine learning and some basic theory. Now these are still important, but they are not enough by themselves.

Today, employers want to see real examples of what a candidate can do. Since AI tools can help people prepare better CVs, cover letters and answers, companies care about projects, GitHub accounts and practical experience. Because of this, students should prepare a stronger portfolio before applying for jobs.

Another important change is that candidates should know how to use generative AI tools correctly. For example, they may use AI for coding research or solving problems, but they also need to understand the output. If a person uses AI and cannot explain the result, this can create a bad impression in the interview.

Interviews have also become more important. Employers may ask technical questions, give coding tasks or ask candidates to explain their projects in detail. This is a way to understand whether the candidate really has the knowledge or only used AI tools without understanding.

Also soft skills are more important now. Skills such as communication, critical thinking and being able to adapt are valuable, because AI cannot fully replace human judgment.

In conclusion, generative AI has made job applications in AI more competitive. Before applying, candidates need to prepare projects and show their skills clearly. During the applications process, they must use AI carefully and also prove that they really understand their work.

1.d.ii) Ask the generative AI the question in reasoning mode and write down what you understand from its answer.

What has changed in terms of what needs to be done during and before job applications in the field of artificial intelligence following developments in the field of generative artificial intelligence?

I'm checking recent sources on hiring, skills, and recruiting changes tied to generative AI, then I'll turn that into a direct answer with citations.

14s boyunca düşündüm >

Generative AI has changed AI job applications in two main ways: **what candidates need to do before applying** and **how they must behave during the application process**.

Before applying, it is no longer enough to only know core AI topics in theory. Employers increasingly expect candidates to show that they can **use generative AI tools practically**, not just talk about them. Recent reports show that AI and big data are among the fastest-growing skills, many employers plan to hire people with specific AI skills, and skill requirements across jobs are shifting quickly because of AI. [World E... +2](#)

This means candidates now need to prepare a stronger portfolio. For AI roles, that usually includes projects showing how they used LLMs, prompt engineering, retrieval-augmented generation, model evaluation, AI safety, or workflow automation. A resume that lists "Python, ML, deep learning" is weaker than one that proves the candidate built and tested systems using current generative AI tools. Recruiters are also becoming more skills-focused because AI is changing hiring and work design itself. [business... +2](#)

Another change is that applicants now need to prepare for more competition and more filtering. Because generative AI helps people write resumes, cover letters, and application answers faster, more candidates can submit polished applications at scale. That raises the baseline. So a generic, AI-written application is less useful now. Candidates need clearer evidence of real ability: measurable project results, GitHub work, published experiments, model comparisons, benchmarks, or case studies. At the same time, recruiters themselves are using AI to speed up screening and hiring workflows. [business... +1](#)

During the application process, authenticity matters more than before. Using generative AI to improve grammar, structure, or formatting is now common. But candidates are increasingly expected to **understand and defend everything they submit**. If an application, coding task, or written answer looks polished but the candidate cannot explain the reasoning in an interview, that becomes a red flag. In other words, generative AI can help you present yourself, but it cannot replace actual understanding. This follows the broader workplace trend that companies want people who can use AI effectively while still applying judgment, verification, and human decision-making. [McKinse... +2](#)

Technical interviews have also shifted. In many cases, employers are less impressed by memorized answers and more interested in whether the candidate can work like a real practitioner: define a problem clearly, choose the right tools, evaluate AI output, detect hallucinations, and improve a system iteratively. Since AI tools are already widely used in development, the differentiator is no longer "Do you use AI?" but "Can you use it well, critically, and responsibly?" [survey.s... +2](#)

So the main change is this: **before GenAI, applicants mostly had to prove knowledge; after GenAI, they must prove judgment, practical tool use, and real output**. Preparation now requires stronger projects, clearer specialization, and evidence that the applicant can work with AI tools without depending on them blindly. During app↓ons, the expectation is not to avoid AI completely, but to use it transparently and intelligently while still showing original thinking and technical depth. [World E... +2](#)

// Explanation

From the AI's answer, I understood that job applications in the AI field are different now because of generative AI. Before, it was mostly enough to know topics like machine learning, deep learning, and programming. Now, companies also want to see that a person can use AI tools in real life, not only know the theory. This means students and job seekers should prepare better projects, a stronger portfolio, and show real examples of their work.

I also understood that many people use AI to write CVs, cover letters, and application answers, so competition is higher now. Because of this, being good at writing with AI is not enough by itself. Companies want candidates who can explain their work clearly and show that they really understand it. So, generative AI is helpful during job applications, but it cannot replace real knowledge, skills, and personal effort.

2) Problem Solving and Coding

2.a Solving the Eight Queen Problem with First-Choice Hill Climbing / Solving the Bridge and Torch Problem with A*

2.a.i Codes and Outputs

```
import random

import time

import matplotlib.pyplot as plt

N = 8

# Heuristic: number of conflicts

def heuristic(board):

    conflicts = 0

    for i in range(N):

        for j in range(i + 1, N):

            # same row

            if board[i] == board[j]:

                conflicts += 1

            # diagonal

            if abs(board[i] - board[j]) == abs(i - j):

                conflicts += 1

    return conflicts
```

```

# Generate random board

def random_board():
    return [random.randint(0, N - 1) for _ in range(N)]

# Get best neighbor

def best_neighbor(board):
    best = board[:]
    best_h = heuristic(board)

    for col in range(N):
        original_row = board[col]

        for row in range(N):
            if row == original_row:
                continue

            new_board = board[:]
            new_board[col] = row

            h = heuristic(new_board)

            if h < best_h:
                best = new_board
                best_h = h

    return best, best_h

# Hill Climbing with Random Restart

def solve():
    current = random_board()
    current_h = heuristic(current)

    moves = 0
    restarts = 0

    while current_h != 0:

```

```

neighbor, neighbor_h = best_neighbor(current)

if neighbor_h < current_h:
    current = neighbor
    current_h = neighbor_h
    moves += 1
else:
    # stuck -> random restart
    current = random_board()
    current_h = heuristic(current)
    restarts += 1

return current, moves, restarts

# Print board
def print_board(board):
    for r in range(N):
        row = ""
        for c in range(N):
            if board[c] == r:
                row += "Q "
            else:
                row += ". "
        print(row)
    print()

# Run 10 trials
def run_trials():
    results = []
    for i in range(10):
        start = time.perf_counter()

```

```

solution, moves, restarts = solve()

end = time.perf_counter()

elapsed = end - start

results.append((moves, restarts, elapsed))

print(f"Trial {i+1}")

print_board(solution)

print("Final Heuristic:", heuristic(solution))

draw_board(solution, i+1)

print(f"Moves: {moves}, Restarts: {restarts}, Time: {elapsed:.6f}s")

print("-" * 40)

return results

# Print table
def print_table(results):

    print("\nFINAL TABLE")

    print("Moves | Restarts | Time (s)")

    print("-----")

    total_moves = 0

    total_restarts = 0

    total_time = 0

    for (m, r, t) in results:

        print(f"{m:5} | {r:8} | {t:.6f}")

        total_moves += m

        total_restarts += r

        total_time += t

    print("-----")

    print(f"{total_moves/10:.2f} | {total_restarts/10:.2f} | {total_time/10:.6f}")

def draw_board(board, trial_num):

```

```
fig, ax = plt.subplots()

# Draw a board (8x8)
for row in range(N):
    for col in range(N):
        color = "#EEEEED" if (row + col) % 2 == 0 else "#769656"
        ax.add_patch(plt.Rectangle((col, row), 1, 1, color=color))

# Put the Queen's
for col in range(N):
    row = board[col]

    ax.text(col + 0.5, row + 0.5, '♔',
            ha='center', va='center', fontsize=20, color='black')

ax.set_xlim(0, N)
ax.set_ylim(0, N)
ax.set_xticks([])
ax.set_yticks([])
ax.set_title(f"Trial {trial_num}")

plt.gca().invert_yaxis()

# save as PNG to put in project template
plt.savefig(f"board_trial_{trial_num}.png")

plt.close()

# MAIN
if __name__ == "__main__":
    results = run_trials()

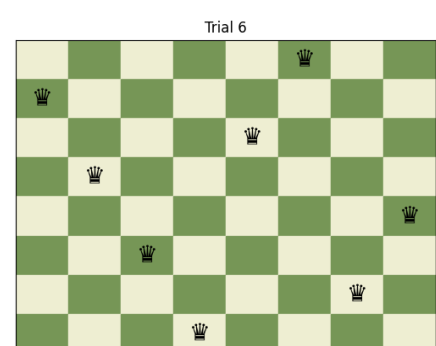
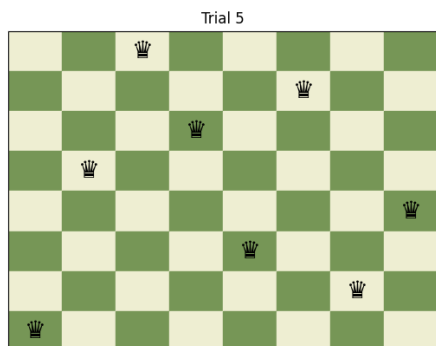
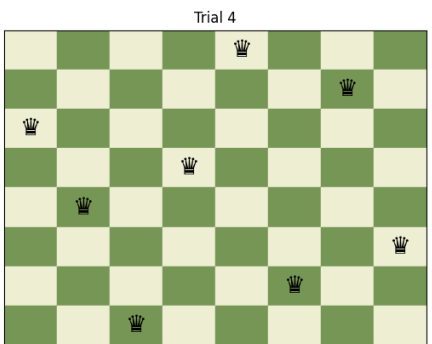
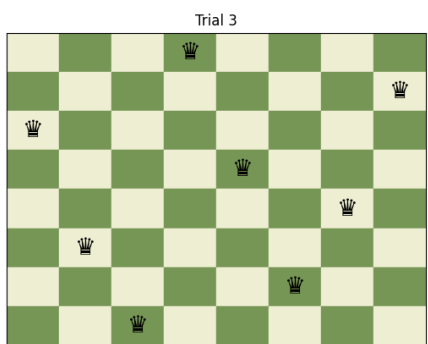
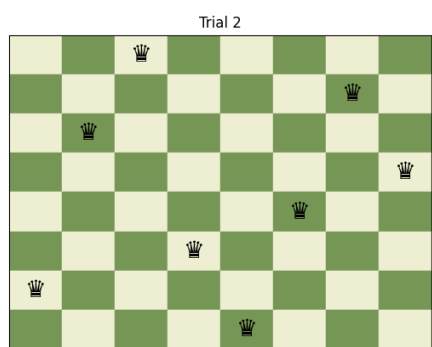
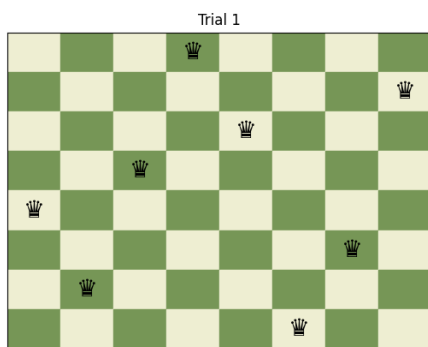
    print_table(results)
```

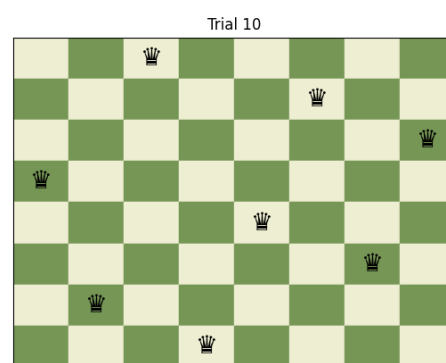
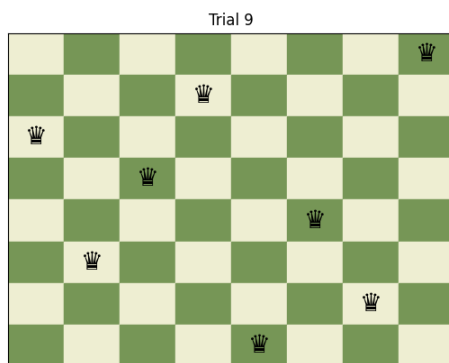
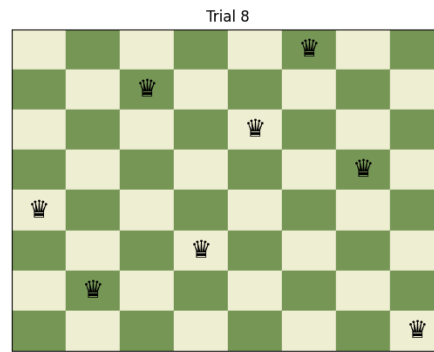
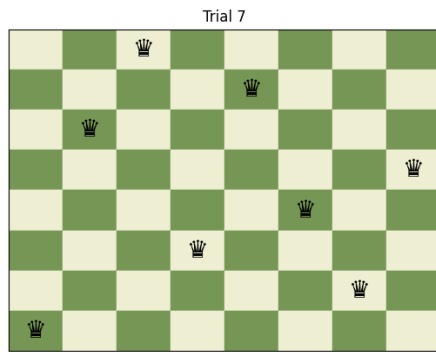
Table with number of relocations, number of random restarts and execution times

Run No	Number of relocations	Number of Random restart	Execution Time
1	11	2	0.002971

2	6	0	0.001349
3	20	5	0.004847
4	7	1	0.001555
5	6	1	0.001366
6	20	5	0.004949
7	4	0	0.000846
8	16	4	0.003897
9	5	1	0.001207
10	102	30	0.025598

//Screenshots showing Board Status





2.a.ii What is Stochastic Hill Climbing? Explain which part of the Hill Climbing source code needs to be changed.

Stochastic Hill Climbing is a variation of the hill climbing algorithm where, instead of always selecting the best possible neighbor, a randomly chosen better neighbor is selected.

In standard hill climbing, all neighboring states are evaluated and the one with the lowest heuristic value (best improvement) is selected. This makes the algorithm greedy and more likely to get stuck in local optima.

In contrast, stochastic hill climbing introduces randomness by selecting one of the improving neighbors at random. This allows the algorithm to explore different paths and reduces the chance of getting trapped in local optima.

To implement stochastic hill climbing in the source code, the main modification should be made in the neighbor selection part of the algorithm.

Instead of the function that selects the best neighbor (`best_neighbor`), we need to:

- Generate all neighbors
- Filter only those neighbors that have a better heuristic value than the current state
- Randomly select one of these better neighbors

If no better neighbor exists, the algorithm performs a random restart as before.

This change affects only the neighbor selection strategy, while the rest of the algorithm (heuristic function, state representation, and restart mechanism) remains the same.

3) Genetic Algorithms

3.a Generated Questions and Your Answers

//Questions and answers

Question-1

In a genetic algorithm solving a Traveling Salesman Problem (TSP), which encoding method best preserves solution validity?

- A) Binary Encoding
- B) Tree Encoding
- C) Value Encoding
- D) Permutation Encoding

Correct Answer: D

Explanation: Permutation encoding ensures each city appears exactly once, preserving constraints

Question-2

In a GA using Roulette Wheel Selection, if one individual has significantly higher fitness than others, what is the most likely outcome?

- A) The algorithm never converges
- B) Exploration increases significantly
- C) One individual dominates and diversity decreases
- D) Mutation rate increases automatically

Correct Answer: C

Explanation: High fitness creates strong selection pressure, reducing diversity.

Question-3

In a GA design, what is the main purpose of using elitism from an auditing perspective?

- A) Increase randomness
- B) Prevent best solution loss
- C) Eliminate mutation effects
- D) Reduce population size

Correct Answer: B

Explanation: Elitism guarantees best individuals survive to next generation.

Question-4

In a scenario where crossover rate is set to 0% and mutation rate is very low, what is the expected behavior?

- A) Rapid convergence
- B) No evolution occurs
- C) Random search
- D) Infinite loop

Correct Answer: B

Explanation: Without crossover and mutation, population cannot evolve.

Question-5

A GA is stuck in a local optimum despite high crossover rate. What is the most logical adjustment?

- A) Decrease population size
- B) Increase mutation rate
- C) Remove selection
- D) Stop algorithm

Correct Answer: B

Explanation: Mutation helps escape local optima.

Question-6

If population diversity is extremely high, what is the likely consequence?

- A) Fast convergence
- B) No meaningful convergence
- C) No mutation needed
- D) Elitism becomes unnecessary

Correct Answer: B

Explanation: Too much diversity prevents convergence.

Question-7

In a GA, if selection pressure is too strong, what problem is most likely to occur?

- A) Infinite loop
- B) Premature convergence
- C) Increased mutation
- D) Reduced computation time

Correct Answer: B

Explanation: Strong pressure reduces diversity quickly.

Question-8

Which scenario best represents poor fitness function design?

- A) All individuals have different scores
- B) Fitness values clearly distinguish good and bad solutions
- C) All individuals receive similar fitness values
- D) Fitness depends on constraints

Correct Answer: C

Explanation: No distinction prevents proper selection.

Question-9

In a GA applied to a constrained optimization problem, what is the auditor's main concern?

- A) Speed of execution
- B) Memory usage
- C) Validity of generated solutions
- D) Number of generations

Correct Answer: C

Explanation: Solutions must satisfy constraints.

Question-10

A GA consistently produces identical solutions across runs. What is the most likely issue?

- A) Too high mutation rate
- B) Too large population
- C) Lack of diversity or randomness
- D) Too many generations

Correct Answer: C

Explanation: Low diversity leads to repeated results.

//Your personal experiences

While solving these questions, I realized that I couldn't just rely on memorizing definitions. I had to actually think about how genetic algorithms behave in different situations.

Some questions were tricky because more than one option looked correct at first. I had to carefully eliminate the wrong answers and think step by step. This helped me understand the logic behind concepts like mutation rate, selection pressure, and diversity more clearly.

This test also made me think more like an evaluator rather than just a learner. Instead of just knowing what each concept means, I started to question what happens if something goes wrong in the algorithm.

Overall, this approach was more useful than just studying notes because it forced me to actively think and analyze. I feel like I understand genetic algorithms better now.

4) Machine Learning with Python [Classifier]

4.a Python Standardization Example

4.a.i Codes

```
import numpy as np

from sklearn.preprocessing import StandardScaler

# Example data
data = np.array([ [10], [20], [30], [40], [50]])

print("Original Data:")

print(data)

scaler = StandardScaler()

standardized_data = scaler.fit_transform(data)

print("\nStandardized Data:")

print(standardized_data)
```

4.a.ii Difference from Normalization

Standardization and normalization are both used to scale data, but they work in different ways. Standardization transforms the data so that it has a mean of 0 and a standard deviation of 1. It does not limit the values to a specific range, but instead centers the data around zero. Normalization, on the other hand, rescales the data to a fixed range, usually between 0 and 1, using the minimum and maximum values of the dataset. In simple terms, standardization adjusts the data based on its distribution, while normalization adjusts it based on its range.

4.b Finding or Creating Dataset

For this project, we selected the Breast Cancer Wisconsin (Diagnostic) dataset from the UCI Machine Learning Repository. This dataset is used to classify tumors as either malignant or benign based on various features extracted from medical images. The source we selected dataset from is UCI Machine Learning Repository. And here is the link:

[https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))

4.c Review the Dataset

The Breast Cancer Wisconsin (Diagnostic) dataset was analyzed in order to understand its structure and characteristics for the classification task.

- Number of Instances: 569
- Number of Features (Attributes): 30

The dataset contains numerical features that are computed from digitized images of breast mass samples. These features describe characteristics of the cell nuclei present in the images. Some of the main attributes include radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, and fractal dimension.

- Number of Classes: 2
- Classes:
 - Malignant (cancerous)
 - Benign (non-cancerous)

Each instance in the dataset represents a tumor sample, and the goal is to classify whether the tumor is malignant or benign based on the given features.

4.d Two different classifier and results

4.d.i Source Code

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import confusion_matrix

from sklearn.ensemble import RandomForestClassifier

from sklearn.neighbors import KNeighborsClassifier

# We are loading our dataset

data = load_breast_cancer()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# Standardization

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

```

# We used Random Forest and KNN as classifiers

rf = RandomForestClassifier()

rf.fit(X_train, y_train)

rf_pred = rf.predict(X_test)

rf_cm = confusion_matrix(y_test, rf_pred)

print("Random Forest Confusion Matrix:")

print(rf_cm)

#And our second classifier KNN

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

knn_pred = knn.predict(X_test)

knn_cm = confusion_matrix(y_test, knn_pred)

print("\nKNN Confusion Matrix:")

print(knn_cm)

```

4.d.ii Confusion Matrices

Our confusion matrices:

```

● ceylintor@192 eight-queens % python3 ml.py
Random Forest Confusion Matrix:
[[40  3]
 [ 1 70]]

KNN Confusion Matrix:
[[40  3]
 [ 3 68]]

```

And we would like to add our opinion: These confusion matrices show that both classifiers perform well. However, the Random Forest classifier has fewer misclassifications compared to KNN, indicating better performance.

4.e Predicting the class of an instance that its class is unknown

4.e.i Source Code

```

# --- New Instance Prediction ---

new_instance = [X_test[0]]

```

```

# We applied the same scaling
new_instance_scaled = scaler.transform(new_instance)

# Predictions
rf_prediction = rf.predict(new_instance_scaled)
knn_prediction = knn.predict(new_instance_scaled)

# Class names
target_names = data.target_names

print("\n--- New Instance Prediction ---")

print(f"Random Forest Prediction: {rf_prediction[0]} -> {target_names[rf_prediction[0]]}")
print(f"KNN Prediction: {knn_prediction[0]} -> {target_names[knn_prediction[0]]}")

```

4.e.ii Output

```

ceylintor@192 eight-queens % python3 4thquestion.py
Random Forest Confusion Matrix:
[[40  3]
 [ 1 70]]

KNN Confusion Matrix:
[[40  3]
 [ 3 68]]

--- New Instance Prediction ---
Random Forest Prediction: 1 -> benign
KNN Prediction: 1 -> benign

```

4.f 3-Fold Cross Validation

4.f.i Source Code

```

from sklearn.model_selection import cross_validate

from sklearn.pipeline import Pipeline

# Define scoring metrics
scoring = ['accuracy', 'precision_macro', 'recall_macro']

# Pipeline for Random Forest
rf_pipeline = Pipeline([

```

```

    ('scaler', StandardScaler()),

    ('rf', RandomForestClassifier())

])

# Pipeline for KNN

knn_pipeline = Pipeline([

    ('scaler', StandardScaler()),

    ('knn', KNeighborsClassifier(n_neighbors=5))

])

# Cross-validation (3-fold)

rf_scores = cross_validate(rf_pipeline, X, y, cv=3, scoring=scoring)

knn_scores = cross_validate(knn_pipeline, X, y, cv=3, scoring=scoring)

# Print results

print("\n--- 3-Fold Cross Validation Results ---")

print("\nRandom Forest:")

print("Accuracy:", rf_scores['test_accuracy'].mean())

print("Precision:", rf_scores['test_precision_macro'].mean())

print("Recall:", rf_scores['test_recall_macro'].mean())

print("\nKNN:")

print("Accuracy:", knn_scores['test_accuracy'].mean())

print("Precision:", knn_scores['test_precision_macro'].mean())

print("Recall:", knn_scores['test_recall_macro'].mean())

```

4.f.ii Classification Performance with 3-Fold Cross Validation

Model	Accuracy	Precision	Recall
Random Forest	0.9578	0.9577	0.9520
KNN	0.9630	0.9664	0.9553

The performance of both classifiers was evaluated using 3-fold cross-validation. The results show that the KNN classifier achieved slightly higher accuracy, precision, and recall compared to the

Random Forest classifier. This indicates that KNN provides better overall performance on this dataset, although the difference between the models is relatively small.

5) Time Spent and Division of labor Details

The project was completed by a team of 5 students. The workload was distributed based on task complexity and contribution levels.

- **Lidya Deliağa:** Worked on **Question 1.a and 1.b** (Algorithms and Game Development using Generative AI) and contributed to report writing. (*Approx. 12 hours*)
- **Ceylin Tor:** Worked on **Question 4 (Machine Learning)** including dataset analysis, classification, prediction, and cross-validation. (*Approx. 13 hours*)
- **Aykut Hacıoğlu:** Worked on **Question 2** (Eight Queens problem and implementation). (*Approx. 9 hours*)
- **Ali Batıkan Öcal:** Worked on **Question 1.c and 1.d** (Definitions, Comparisons, and Job Applications in AI). (*Approx. 9 hours*)
- **Ömer Faruk Çakmakçı:** Worked on **Question 3 (Genetic Algorithms)** and assisted in report editing and organization. (*Approx. 7 hours*)

All members contributed to reviewing, editing, and finalizing the report.

6) Self-assessment Table

	Desired Feature	Exists	Explanation	Predicted Grade
1a	Algorithms and Comparison (10)	☒	A* algorithm was written and compared with Differential Evolution in terms of working principles and application areas.	10/10
1b	Developing a Game Using Gen AI (10)	☒	A 2D game was generated using Gen AI, tested, and improved with additional features such as UI and gameplay enhancements.	10/10
1c	Definition and Comparison (10)	☒	Concepts such as GPT, Gradient Descent, and	10/10

			Nonlinear Problems were explained and compared with appropriate examples.	
1d	Changes in Job Applications in AI (10)	☒	The impact of generative AI on job applications was analyzed and explained based on personal understanding.	10/10
2	Problem Solving and Coding (10)	☒	Eight Queens problem was solved using hill climbing with random restart, and performance metrics such as moves, restarts, and execution time were recorded.	10/10
3	Genetic Algorithms 'evaluator/auditor' (15)	☒	A 10-question test was generated, answered, and analyzed to evaluate understanding of genetic algorithms.	14/15
4	Machine Learning (15)	☒	Classification was implemented using Random Forest and KNN. Prediction and 3-fold cross-validation were applied using scikit-learn, and results were analyzed.	15/15
5	Task Distribution Details (5)	☒	Task distribution and time allocation among team members	5/5

			were clearly defined and explained.	
6	Self-assessment Table + Report (15)	☒	The report was completed according to the template and includes all required sections in a structured format.	14/15
Total Score out of 100:				98/100

In the description section, information about whether it was done, or not done and which items were done and how they were done or why they could not be done should be written briefly. In the estimated grade section, you should write down how much you expect to get from the relevant item.

Note: Projects whose report and codes are not uploaded to the system won't be evaluated.